

SIMATS ENGINEERING
ASSESSMENT TOOL 1 - High (INDUSTRY PROBLEM SOLVING TASKS)

COURSE CODE: CSA0921

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO4: *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3,BL5)*

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks: 50

ANNEXURE A

INDUSTRY PROBLEM SOLVING TASKS

1. Smart Attendance System (BTL: BL3 – Apply)

Develop a GUI-based attendance system for an organization using Swing components. The system should allow marking attendance, viewing reports, and updating records. Use event handling to process user inputs and actions. Implement validation to prevent duplicate entries and incorrect data. Apply suitable layout managers for structured display. Analyze system responsiveness during bulk data entry and suggest enhancements for better performance and usability.

Code of Conduct:

*I **BHEESHMA L / 192511332** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
BHEESHMA L

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Smart Attendance System

Source Code:

```
1 import javax.swing.*;
2 import javax.swing.table.DefaultTableModel;
3 import java.awt.*;
4 import java.awt.event.*;
5 import java.util.HashSet;
6
7 public class SmartAttendance extends JApplet implements ActionListener {
8
9     // Input components
10    JTextField nameField, idField, dateField;
11    JComboBox<String> statusBox;
12
13    // Table and buttons
14    JTable table;
15    DefaultTableModel model;
16    JButton markBtn, updateBtn, reportBtn, clearBtn;
17
18    // Used to prevent duplicate entries
19    HashSet<String> attendanceIds = new HashSet<String>();
20
21    // Main Initialization
22    public void init() {
23        setSize(950, 600);
24        setLayout(new BorderLayout(10, 10));
25
26        // ----- HEADER -----
27        JLabel title = new JLabel(
28            "SMART ATTENDANCE SYSTEM",
29            SwingConstants.CENTER
30        );
31
32        title.setFont(new Font("Arial", Font.BOLD, 25));
33        title.setForeground(Color.WHITE);
34        title.setOpaque(true);
35        title.setBackground(new Color(48, 78, 120));
36        title.setBorder(BorderFactory.createEmptyBorder(15, 10, 15, 10));
37
38        add(title, BorderLayout.NORTH);
39
40        // ----- INPUT PANEL -----
41        JPanel inputPanel = new JPanel(new GridLayout(2, 4, 10, 10));
42        inputPanel.setBorder(
43            BorderFactory.createTitledBorder("Attendance Entry")
44        );
45
46        inputPanel.add(new JLabel("Student ID"));
47        idField = new JTextField();
48        inputPanel.add(idField);
49
50        inputPanel.add(new JLabel("Student Name"));
51        nameField = new JTextField();
52        inputPanel.add(nameField);
53
54        inputPanel.add(new JLabel("Date"));
55        dateField = new JTextField("20-08-2026");
56        inputPanel.add(dateField);
57
58        inputPanel.add(new JLabel("Status"));
59        statusBox = new JComboBox<String>{
60            new String[]{"Present", "Absent"}
61        };
62        inputPanel.add(statusBox);
63
64        // ----- BUTTON PANEL -----
65        JPanel buttonPanel = new JPanel(
66            new FlowLayout(FlowLayout.CENTER, 12, 8)
67        );
68
69        markBtn = new JButton("Mark Attendance");
70        updateBtn = new JButton("Update");
71        reportBtn = new JButton("View Report");
72        clearBtn = new JButton("Clear");
73
74        buttonPanel.add(markBtn);
75        buttonPanel.add(updateBtn);
76        buttonPanel.add(reportBtn);
77        buttonPanel.add(clearBtn);
78
79        // ----- TOP PANEL -----
80        JPanel topPanel = new JPanel(new BorderLayout());
81        topPanel.add(inputPanel, BorderLayout.CENTER);
82        topPanel.add(buttonPanel, BorderLayout.SOUTH);
83
84        add(topPanel, BorderLayout.CENTER);
85
86        // ----- TABLE -----
87        String[] columns = {
88            "Student ID",
89            "Student Name",
90            "Date",
91            "Status"
92        };
93
94        model = new DefaultTableModel(columns, 0) {
95
96            // Prevent direct editing of table cells
97            public boolean isCellEditable(int row, int column) {
98                return false;
99            }
100        };
101    }
```

```

1      table = new JTable(model);
2      table.setRowHeight(28);
3      table.getTableHeader().setFont(
4          new Font("Arial", Font.BOLD, 14)
5      );
6
7      JScrollPane scrollPane = new JScrollPane(table);
8
9      JPanel tablePanel = new JPanel(new BorderLayout());
10     tablePanel.setBorder(
11         BorderFactory.createTitledBorder("Attendance Records")
12     );
13
14     tablePanel.add(scrollPane, BorderLayout.CENTER);
15
16     add(tablePanel, BorderLayout.SOUTH);
17
18     // Make table larger
19     tablePanel.setPreferredSize(new Dimension(900, 260));
20
21     // ----- EVENT HANDLING -----
22     markBtn.addActionListener(this);
23     updateBtn.addActionListener(this);
24     reportBtn.addActionListener(this);
25     clearBtn.addActionListener(this);
26
27     // Select table row when clicked
28     table.addMouseListener(new MouseAdapter() {
29         public void mouseClicked(MouseEvent e) {
30
31             int row = table.getSelectedRow();
32
33             if (row >= 0) {
34                 idField.setText(
35                     model.getValueAt(row, 0).toString()
36                 );
37
38                 nameField.setText(
39                     model.getValueAt(row, 1).toString()
40                 );
41
42                 dateField.setText(
43                     model.getValueAt(row, 2).toString()
44                 );
45
46                 statusBox.setSelectedItem(
47                     model.getValueAt(row, 3).toString()
48                 );
49             }
50         }
51     });
52 }
53
54 // ----- EVENT HANDLER -----
55 public void actionPerformed(ActionEvent e) {
56
57     // Mark attendance
58     if (e.getSource() == markBtn) {
59
60         String id = idField.getText().trim();
61         String name = nameField.getText().trim();
62         String date = dateField.getText().trim();
63         String status = statusBox.getSelectedItem().toString();
64
65         // Validate empty fields
66         if (id.equals("") || name.equals("") || date.equals("")) {
67             JOptionPane.showMessageDialog(
68                 this,
69                 "Please enter all details!",
70                 "Validation Error",
71                 JOptionPane.ERROR_MESSAGE
72             );
73             return;
74         }
75
76         // Validate ID
77         if (!id.matches("[0-9]+")) {
78             JOptionPane.showMessageDialog(
79                 this,
80                 "Student ID must contain numbers only!",
81                 "Invalid Data",
82                 JOptionPane.ERROR_MESSAGE
83             );
84             return;
85         }
86
87         // Validate duplicate
88         if (attendanceIds.contains(id + "_" + date)) {
89
90             JOptionPane.showMessageDialog(
91                 this,
92                 "Attendance already marked for this student on this date!",
93                 "Duplicate Entry",
94                 JOptionPane.WARNING_MESSAGE
95             );
96
97             return;
98         }
99
100    }

```

```

1      // Add record
2      model.addRow(
3          new Object[]{id, name, date, status}
4      );
5
6      attendanceIds.add(id + "_" + date);
7
8      JOptionPane.showMessageDialog(
9          this,
10         "Attendance marked successfully!"
11     );
12
13     clearFields();
14 }
15
16 // Update record
17 else if (e.getSource() == updateBtn) {
18
19     int row = table.getSelectedRow();
20
21     if (row == -1) {
22         JOptionPane.showMessageDialog(
23             this,
24             "Select a record to update!"
25         );
26         return;
27     }
28
29     String id = idField.getText().trim();
30     String name = nameField.getText().trim();
31     String date = dateField.getText().trim();
32     String status = statusBox.getSelectedItem().toString();
33
34     if (id.equals("") || name.equals("") || date.equals("")) {
35         JOptionPane.showMessageDialog(
36             this,
37             "All fields are required!"
38         );
39         return;
40     }
41
42     model.setValueAt(id, row, 0);
43     model.setValueAt(name, row, 1);
44     model.setValueAt(date, row, 2);
45     model.setValueAt(status, row, 3);
46
47     JOptionPane.showMessageDialog(
48         this,
49         "Attendance record updated!"
50     );
51
52     clearFields();
53 }
54
55 // View report
56 else if (e.getSource() == reportBtn) {
57
58     int total = model.getRowCount();
59     int present = 0;
60     int absent = 0;
61
62     for (int i = 0; i < total; i++) {
63
64         String status =
65             model.getValueAt(i, 3).toString();
66
67         if (status.equals("Present"))
68             present++;
69         else
70             absent++;
71     }
72
73     JOptionPane.showMessageDialog(
74         this,
75         "ATTENDANCE REPORT\n\n" +
76         "Total Records : " + total +
77         "\nPresent      : " + present +
78         "\nAbsent       : " + absent,
79         "Attendance Report",
80         JOptionPane.INFORMATION_MESSAGE
81     );
82 }
83
84 // Clear fields
85 else if (e.getSource() == clearBtn) {
86     clearFields();
87 }
88 }
89
90 // Clear input fields
91 void clearFields() {
92     idField.setText("");
93     nameField.setText("");
94     dateField.setText("20-08-2026");
95     statusBox.setSelectedIndex(0);
96     table.clearSelection();
97 }
98 }

```

Output:

Applet Viewer: SmartAttendance.class

Applet

SMART ATTENDANCE SYSTEM

Attendance Entry

Student ID: 124 Student Name: Gokul

Date: 20-08-2026 Status: Present

Attendance Records

Student ID	Student Name	Date	Status
123	Sathya	20-08-2026	Present
121	Krishna	20-08-2026	Absent

Applet started.

Applet Viewer: SmartAttendance.class

Applet

SMART ATTENDANCE SYSTEM

Attendance Entry

Student ID: Student Name: Date: 20-08-2026 Status: Present

Attendance Records

Student ID	Student Name	Date	Status
123	Sathya		Present
121	Krishna	20-08-2026	Absent
124	Gokul	20-08-2026	Present

Attendance Report

Total Records : 3
Present : 2
Absent : 1

Applet started.